

Homework 2

DATA 624

Kevin Havis

September 11, 2025

Table of contents

1 Homework 2 - Chapter 3 Exercises	1
1.1 Exercise 3.1	1
1.1.1 Response	2
1.2 Exercise 3.2	3
1.3 Exercise 3.2	3
1.3.1 Response	3
1.4 Exercise 3.3	10
1.4.1 Response	10
1.5 Exercise 3.4	13
1.5.1 Response	13
1.6 Exercise 3.5	15
1.6.1 Response	15
1.7 Exercise 3.6	19
1.7.1 Response	19
1.8 Exercise 3.7	21
1.9 Exercise 3.8	25
1.9.1 Response	25
1.10 Exercise 3.9	26

1 Homework 2 - Chapter 3 Exercises

1.1 Exercise 3.1

Consider the GDP information in `global_economy`. Plot the GDP per capita for each country over time. Which country has the highest GDP per capita? How has this changed over time?

1.1.1 Response

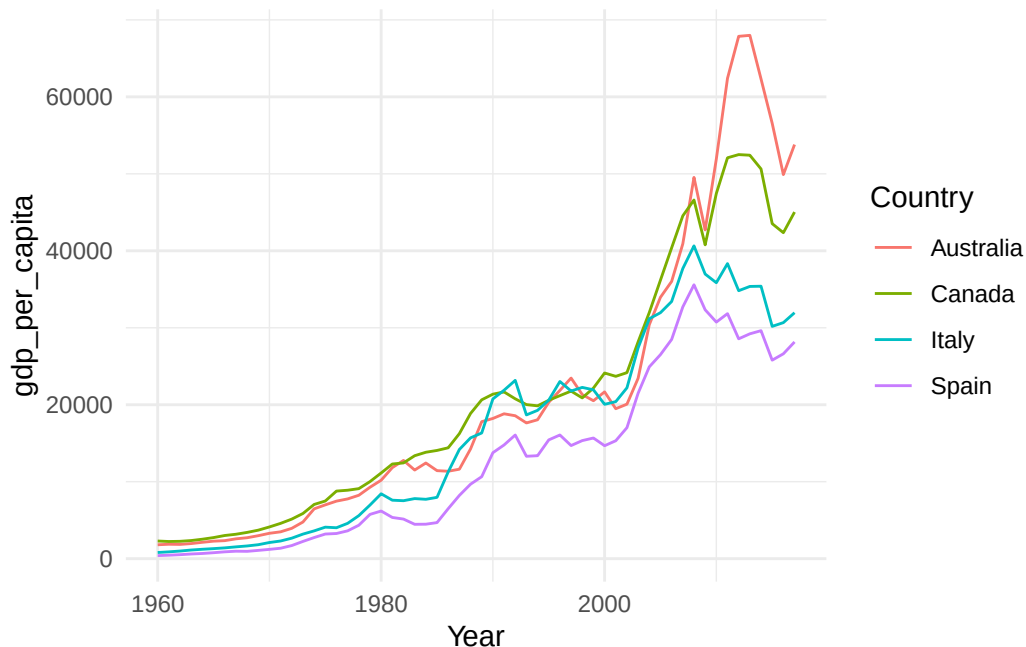
I subset the data to make it more manageable for visualization purposes. In my subset, the highest GDP per capita is by far Australia, which has grown steadily, rapidly peaking in 2012 and rapidly descending afterwards.

```
# There's 250+ countries so limiting for plotting purposes
global_economy_filt <- global_economy |>
  mutate(gdp_per_capita = GDP / Population) |>
  filter(Country %in% c("Italy", "Spain", "Australia", "Canada"))

print(global_economy_filt |> filter(gdp_per_capita == max(gdp_per_capita)))
```

```
# A tibble: 1 x 10 [1Y]
# Key:      Country [1]
  Country   Code  Year      GDP Growth  CPI Imports Exports Population
<fct>     <fct> <dbl>    <dbl> <dbl> <dbl>   <dbl>   <dbl>   <dbl>
1 Australia AUS   2013 1.57e12  2.64  108.    21.3    20.0   23145901
# i 1 more variable: gdp_per_capita <dbl>
```

```
ggplot(global_economy_filt, aes(y=gdp_per_capita, x=Year, color=Country)) +
  geom_line()
```



1.2 Exercise 3.2

1.3 Exerise 3.2

For each of the following series, make a graph of the data. If transforming seems appropriate, do so and describe the effect. * United States GDP from `global_economy`. * Slaughter of Victorian “Bulls, bullocks and steers” in `aus_livestock`. * Victorian Electricity Demand from `vic_elec`. * Gas production from `aus_production`.

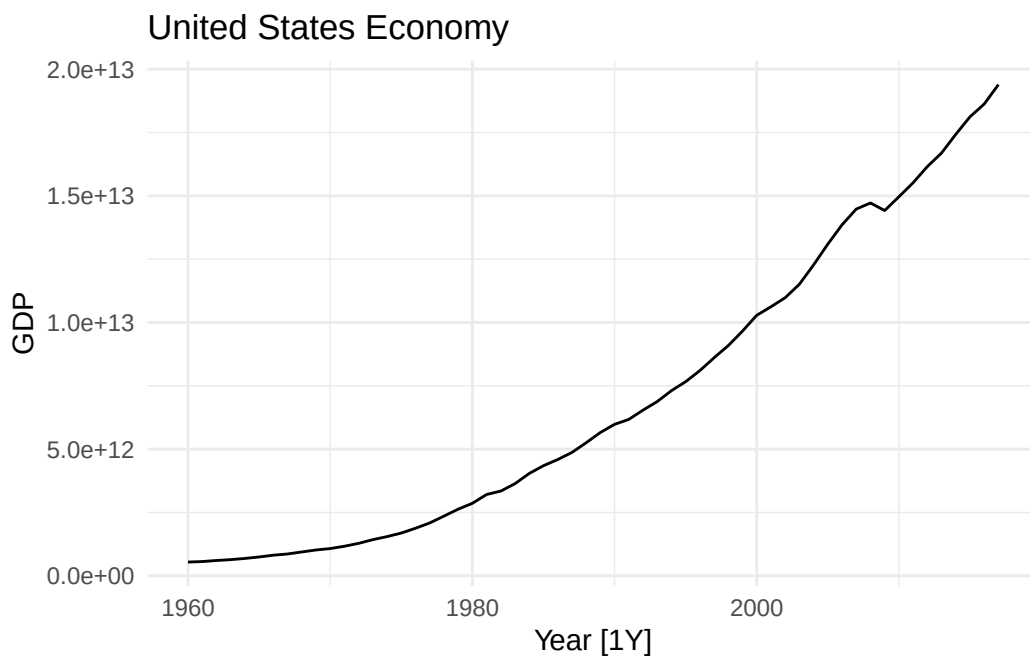
1.3.1 Response

1.3.1.1 U.S. Economy

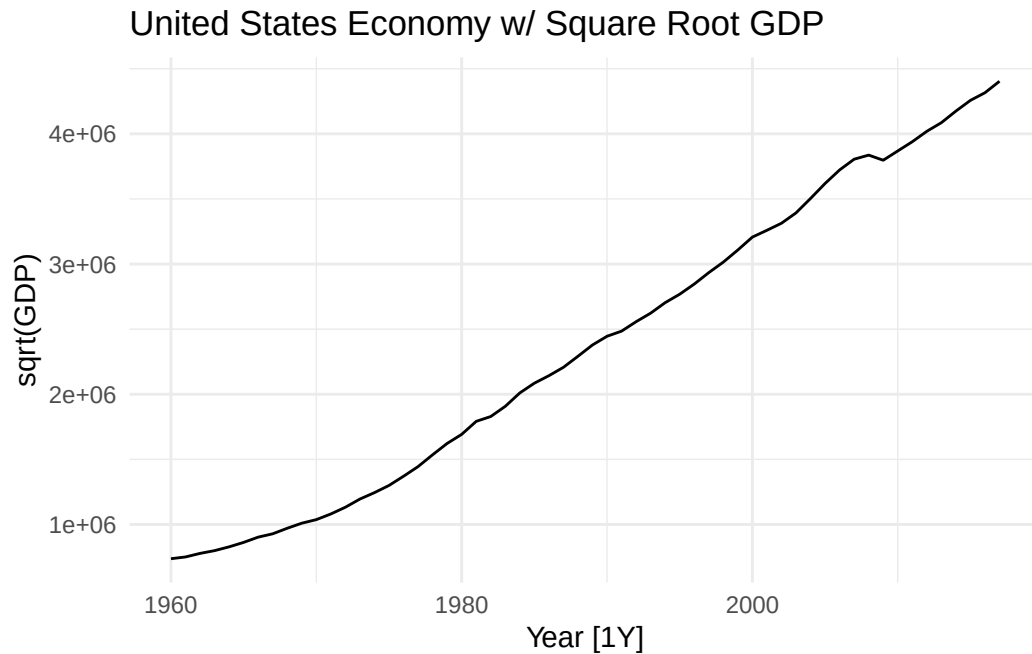
The U.S. Economy data appeared to be slightly quadratic, so I applied a square root on the GDP to get a more linear result. We also could a apply a per-capita effect if desired, useful when comparing to other countries.

```
# US Econ
us_econ <- global_economy |>
  filter(Code == "USA")

autoplot(us_econ) + labs(title = "United States Economy")
```



```
autoplot(us_econ, sqrt(GDP)) + labs(title="United States Economy w/ Square Root GDP")
```

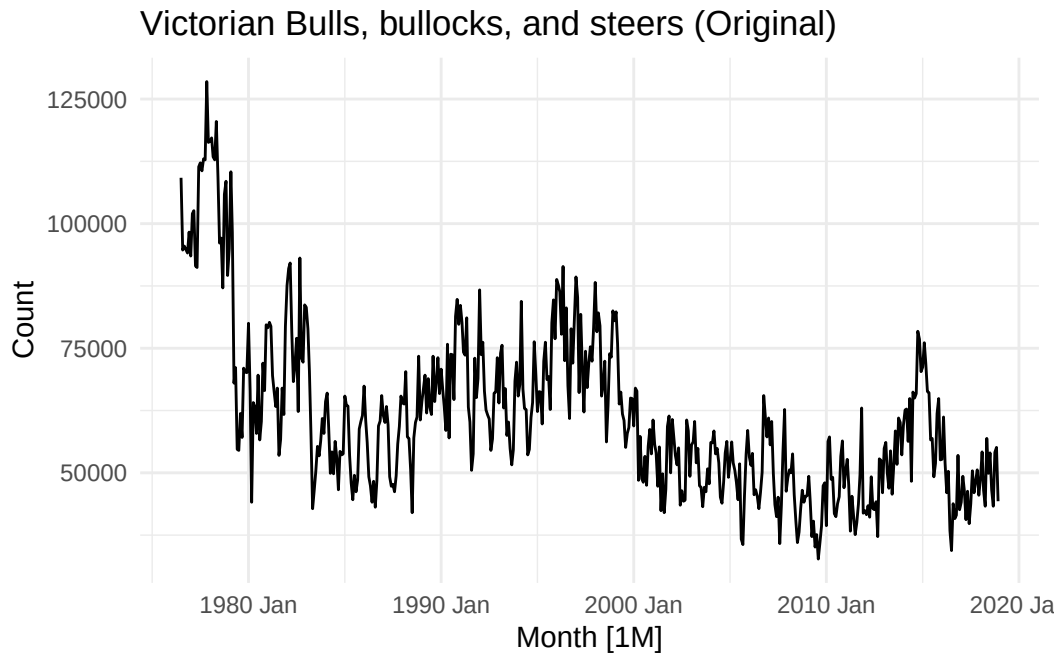


1.3.1.2 Beef Livestock

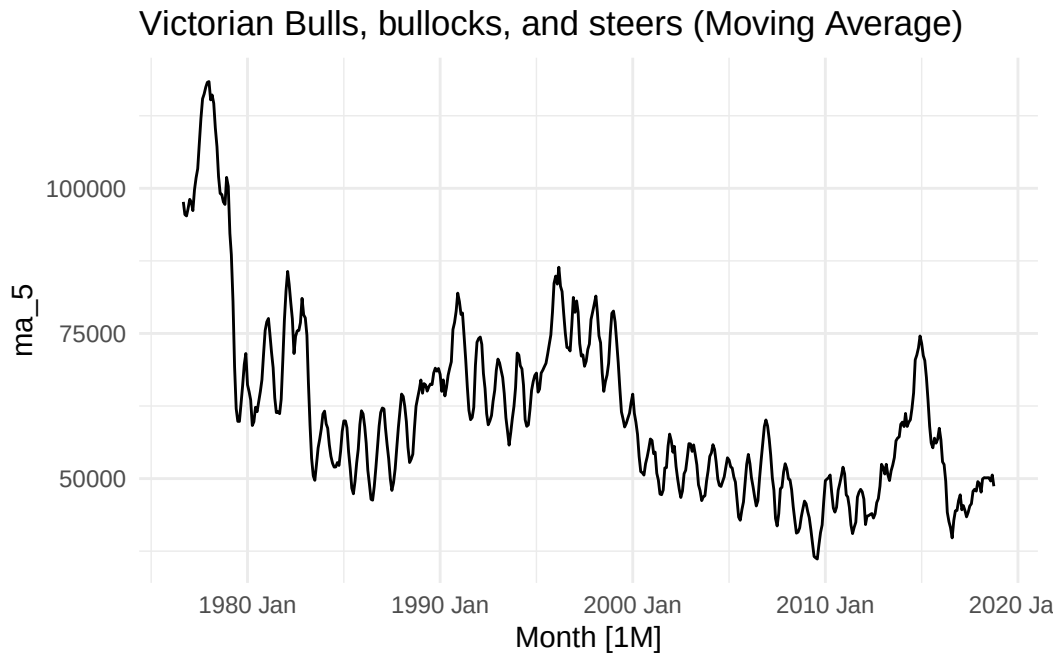
This timeseries is quite busy with many month-to-month variations. I have applied a simple five month moving average to help smooth the overall pattern, which still reveals seasonality, but makes it more clear and the trend more obvious.

```
# Livestock
beef <- aus_livestock |>
  filter(Animal == "Bulls, bullocks and steers") |>
  filter(State == "Victoria")

# A very noisy time series
autoplot(beef) + labs(title="Victorian Bulls, bullocks, and steers (Original)")
```



```
# Use basic moving average smoothing
smooth_beef <- beef |>
  arrange(Month) |>
  mutate(
    ma_5 = slide_dbl(Count, mean, .before=2, .after=2, .complete=TRUE)
  )
autoplot(smooth_beef, ma_5) + labs(title="Victorian Bulls, bullocks, and steers (Moving Average)")
```



1.3.1.3 Victoria Electricity Demand

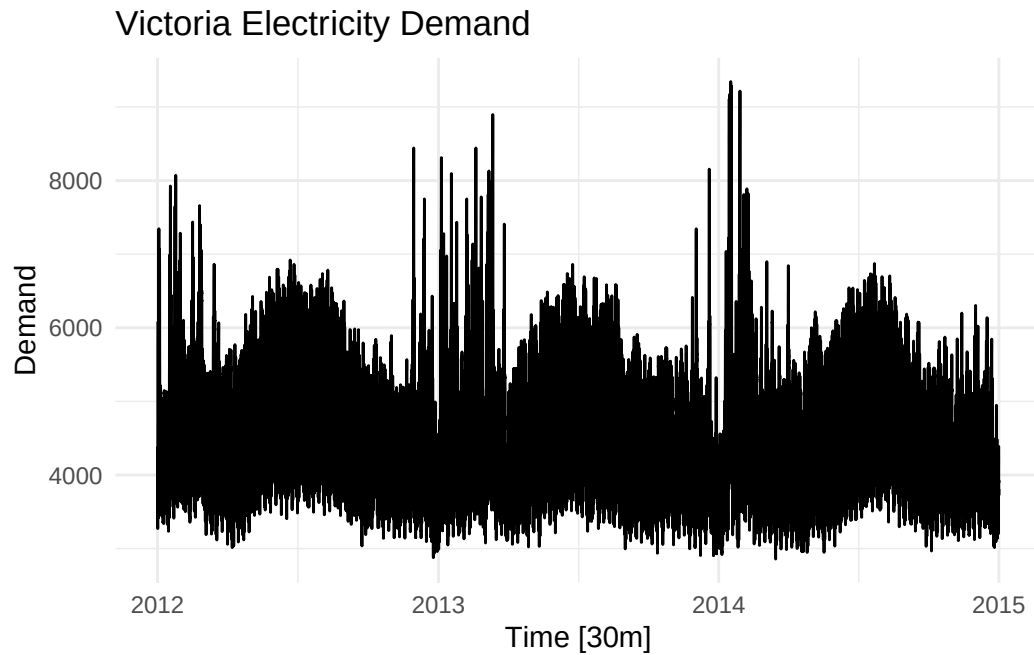
This is the most detailed data we have seen yet; the data represents electricity demand down to the half hour.

While there are likely interesting observations in such granular data, we could use a moving average of the demand to eliminate the interday seasonality, looking back and forward by 12 hours (24 half hour intervals).

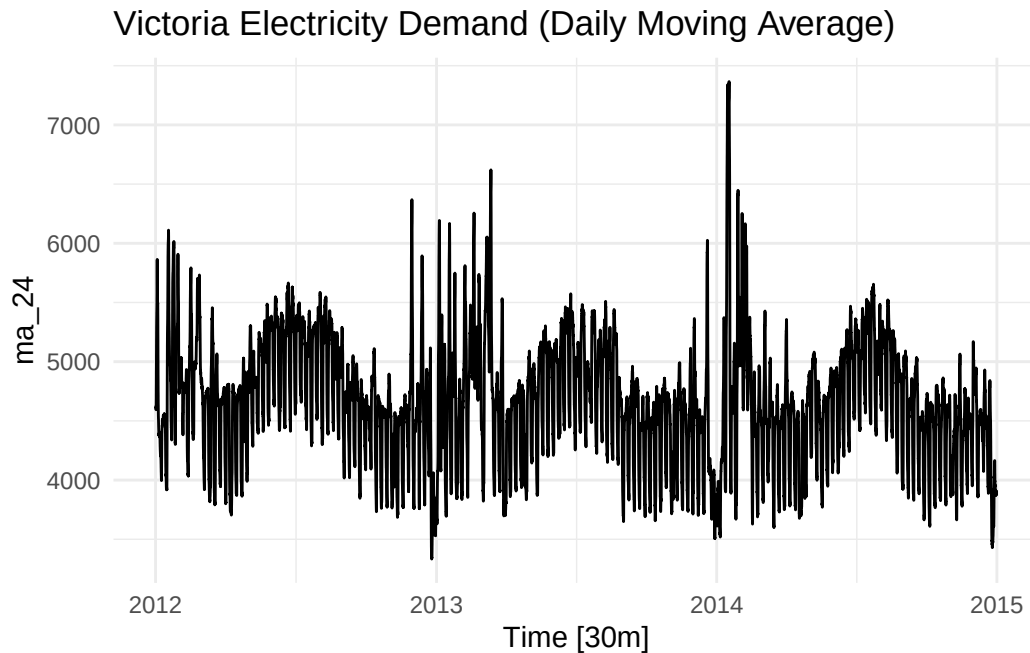
While still a very visually dense graph, this transformation it “thins” the series enough to be more digestible.

These types of transformations demand on the objective, and instead of transforming, decomposition would be more generally useful to extract different levels of seasonality.

```
# Vic Elec Demand
vic_elec |>
  autoplot() + labs(title="Victoria Electricity Demand")
```



```
vic_elec |>
  arrange(Time) |>
  mutate(ma_24 = slide_dbl(Demand, mean, .before =24, .after=24, .complete=TRUE)) |>
  autoplot(ma_24) + labs(title="Victoria Electricity Demand (Daily Moving Average)")
```



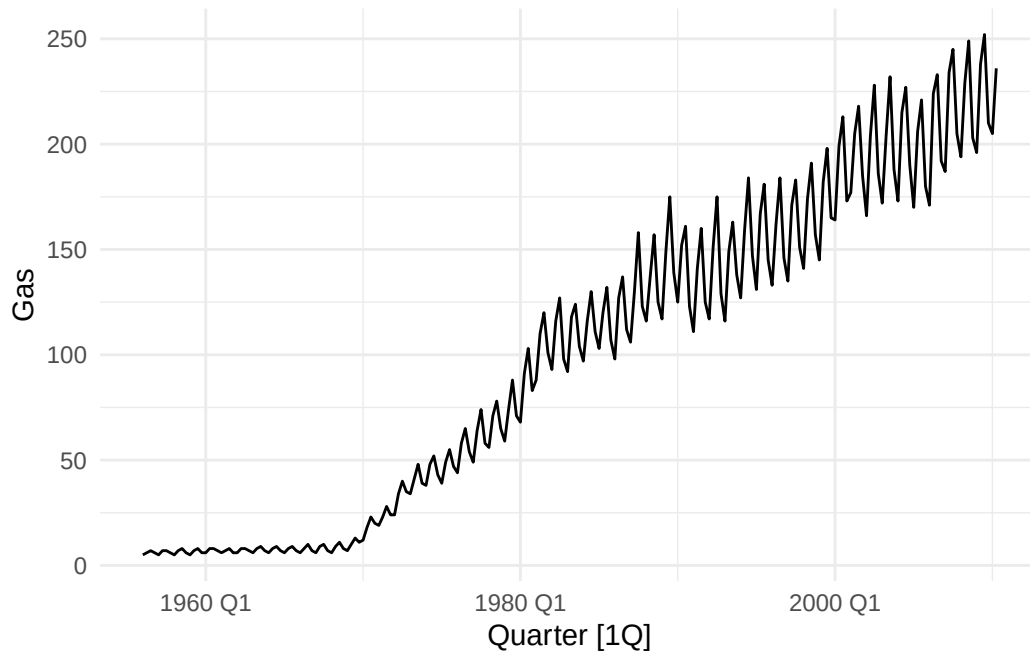
1.3.1.4 AUS Production

Australian gas production has increased but become more varied as time has gone on. This is a great use case for a Box Cox transformation to help “squish” the variance to a regularized amount to make the time series more comparable over time.

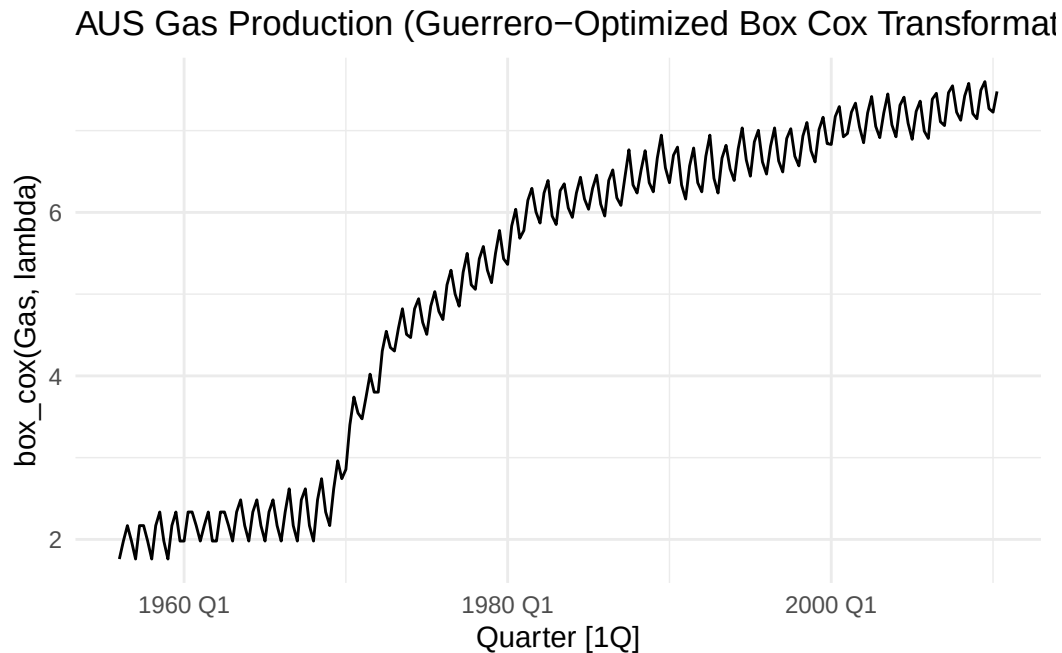
We can use the `guerrero` optimizer to find a value of λ (the Box Cox parameter) that works well for our data.

Upon transforming, we can see our time series has a much more consistent level of variance over time, making further analysis easier. We must remember to reverse the transformation before interpreting though!

```
# AUS Production
aus_production |>
  autoplot(Gas)
```

```
# Straight from the textbook
lambda <- aus_production |>
  features(Gas, features = guerrero) |>
  pull(lambda_guerrero)
aus_production |>
  autoplot(box_cox(Gas, lambda)) +
  labs(title="AUS Gas Production (Guerrero-Optimized Box Cox Transformation)")
```



1.4 Exercise 3.3

Why is a Box-Cox transformation unhelpful for the `canadian_gas` data?

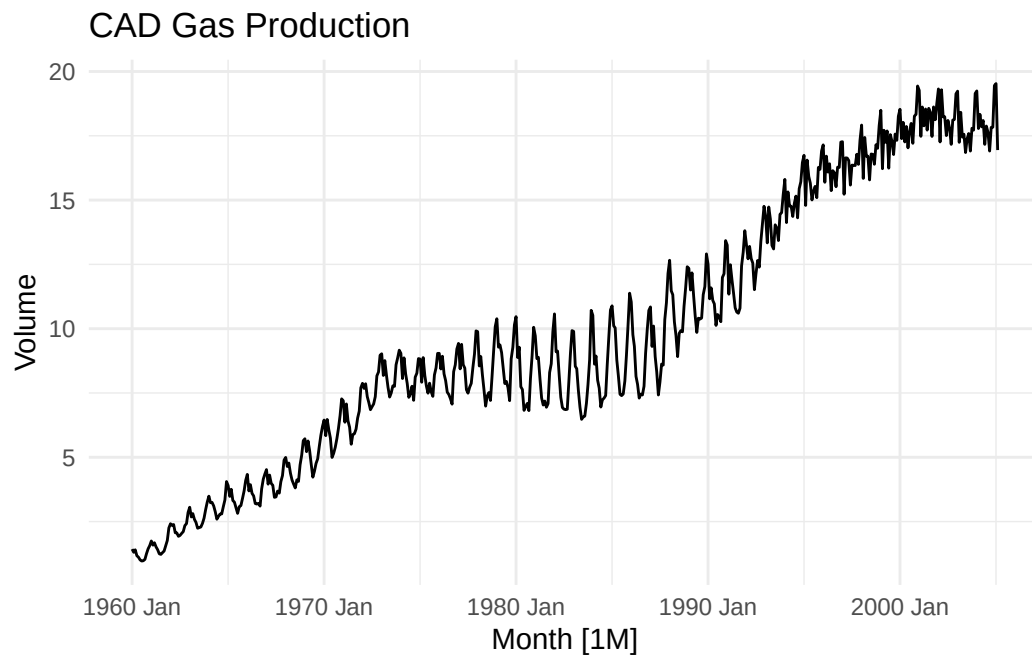
1.4.1 Response

Box Cox transformations can be helpful when the variance of a series trends in one direction or another with some consistency.

Reviewing the `canadian_gas` series, the variance starts small, increases through the middle of the series, and then becomes small yet more erratic in the recent data. Applying a Box Cox transformation would apply the same effect (compressing or expanding) to *all points* of the time series, meaning the transformation would do a poor job normalizing this series.

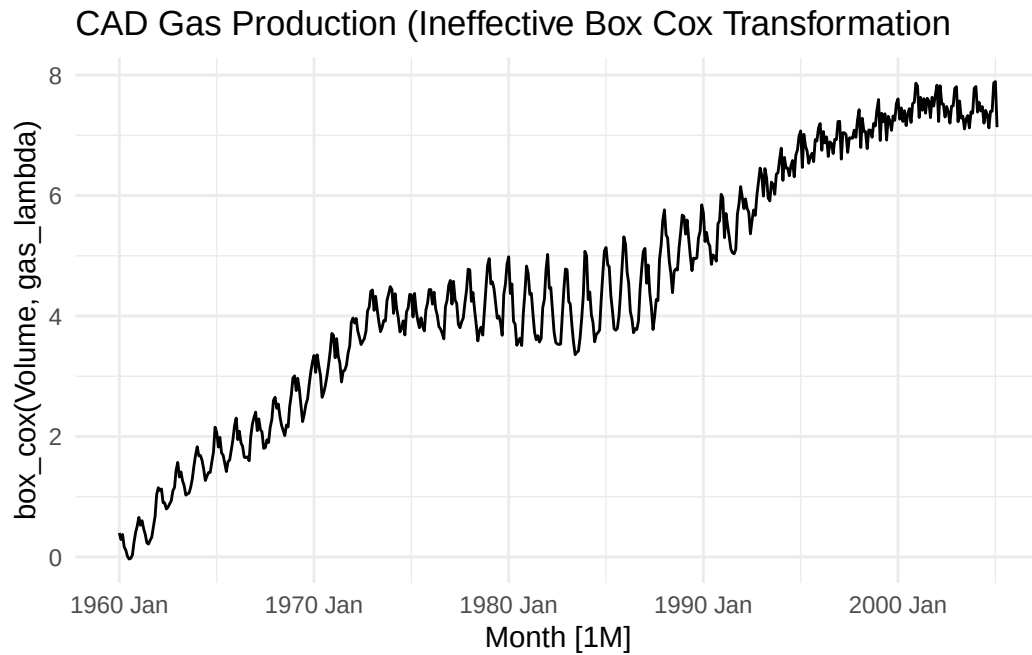
A better option would be Loess smoothing, which uses localized data to dynamically smooth the curve.

```
canadian_gas |>  
  autoplot() + labs(title="CAD Gas Production")
```

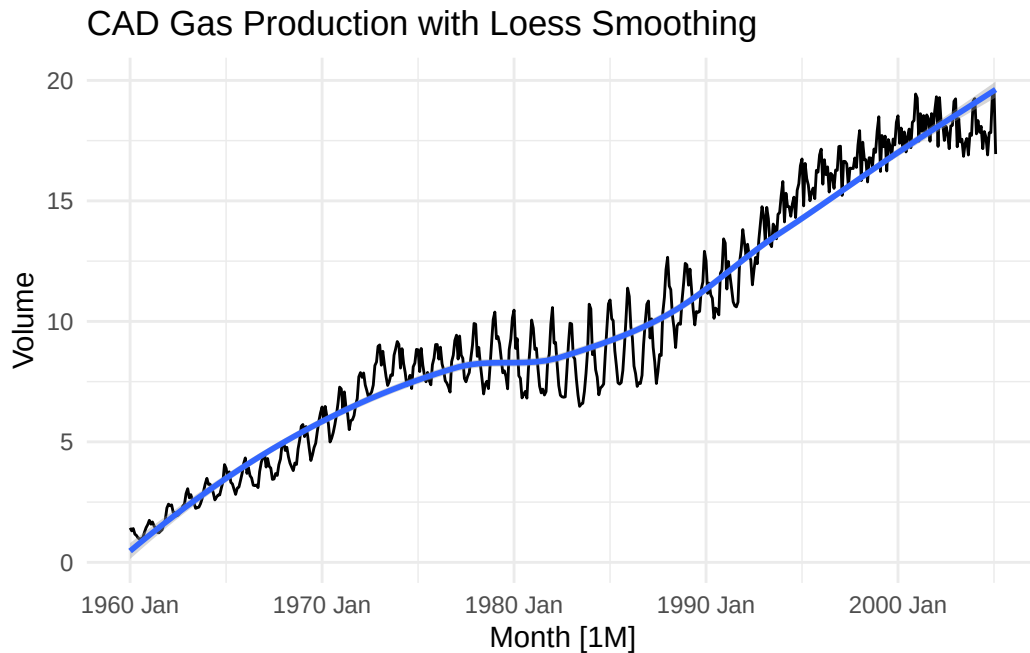


```
gas_lambda <- canadian_gas |>
  features(Volume, features = guerrero) |>
  pull(lambda_guerrero)

canadian_gas |>
  autoplot(box_cox(Volume, gas_lambda)) +
  labs(title="CAD Gas Production (Ineffective Box Cox Transformation)")
```



```
canadian_gas |>  
  autoplot() +  
  geom_smooth() +  
  labs(title="CAD Gas Production with Loess Smoothing")
```



1.5 Exercise 3.4

What Box-Cox transformation would you select for your retail data (from Exercise 7 in Section 2.10)?

1.5.1 Response

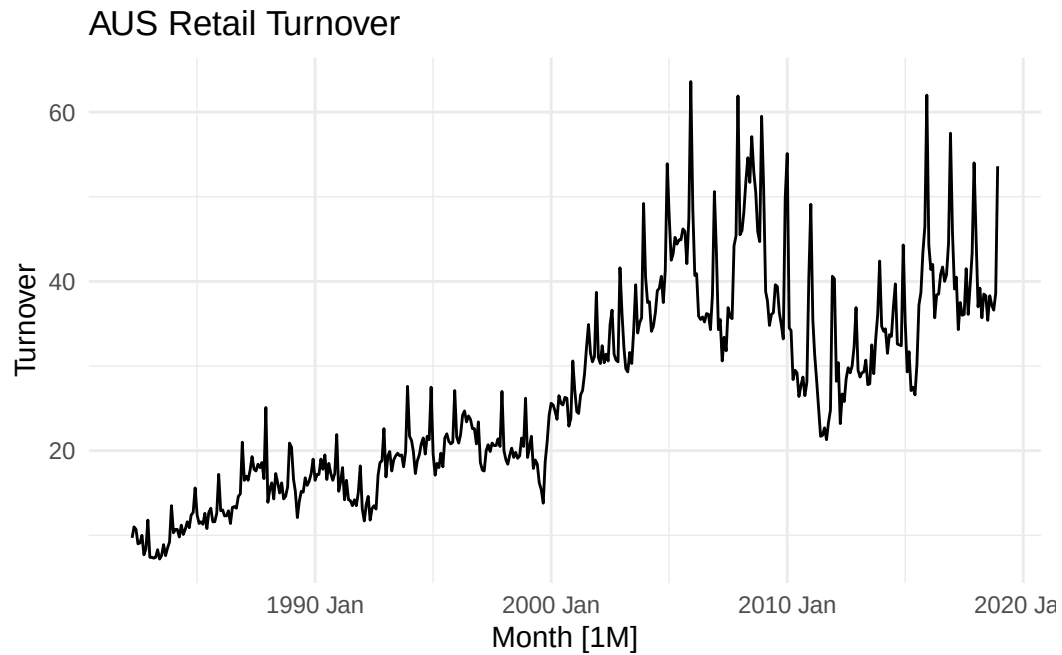
The variance of this time series grows quite quickly, and as such we will need a more intense transformation.

A Box Cox transformation with $\lambda = 0$ gives us the desired effect. This is a log transformation, and once applied, we can see that it has effectively standardized the variance across the series.

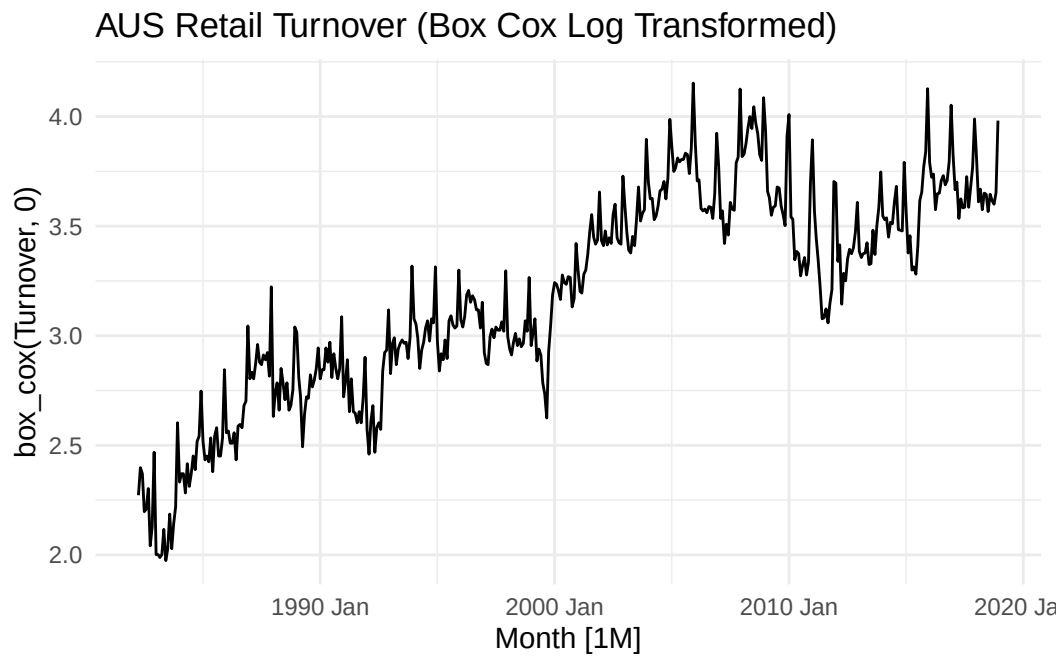
```
set.seed(42)

myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

autoplot(myseries) + labs(title="AUS Retail Turnover")
```



```
autoplot(myseries, box_cox(Turnover, 0)) + labs(title="AUS Retail Turnover (Box Cox Log Trans")
```



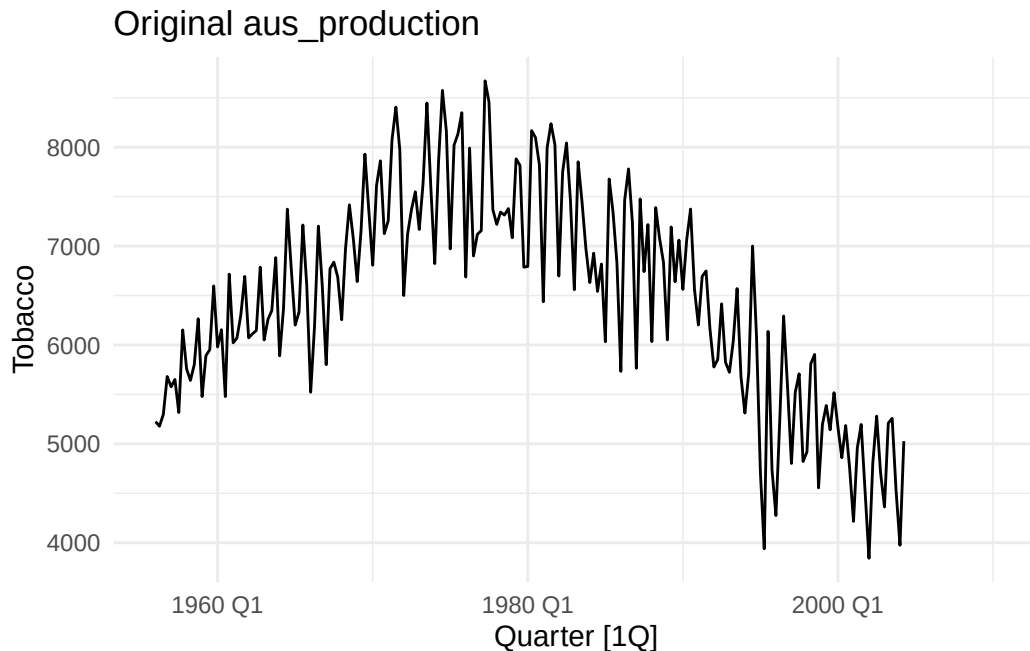
1.6 Exercise 3.5

For the following series, find an appropriate Box-Cox transformation in order to stabilise the variance. * Tobacco from `aus_production` * Economy class passengers between Melbourne and Sydney from `ansett` * Pedestrian counts at Southern Cross Station from `pedestrian`

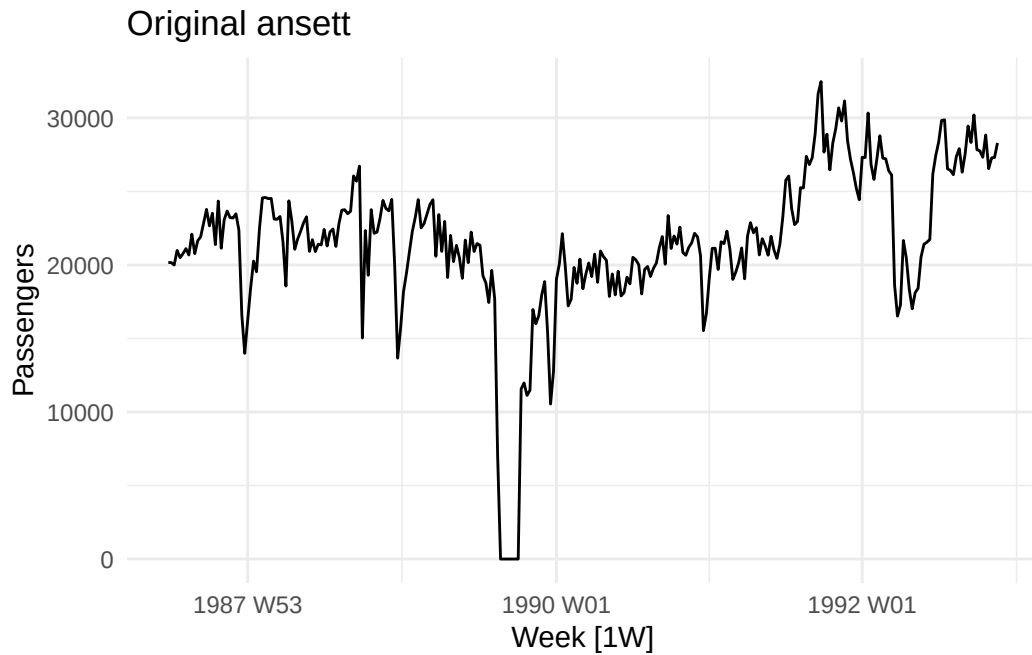
1.6.1 Response

Instead of eyeballing and doing trial by error for each series, we will write a function that uses the `guerrero` optimizer. This optimizer will find the ideal value of λ that will normalize the variance via a Box Cox transformation.

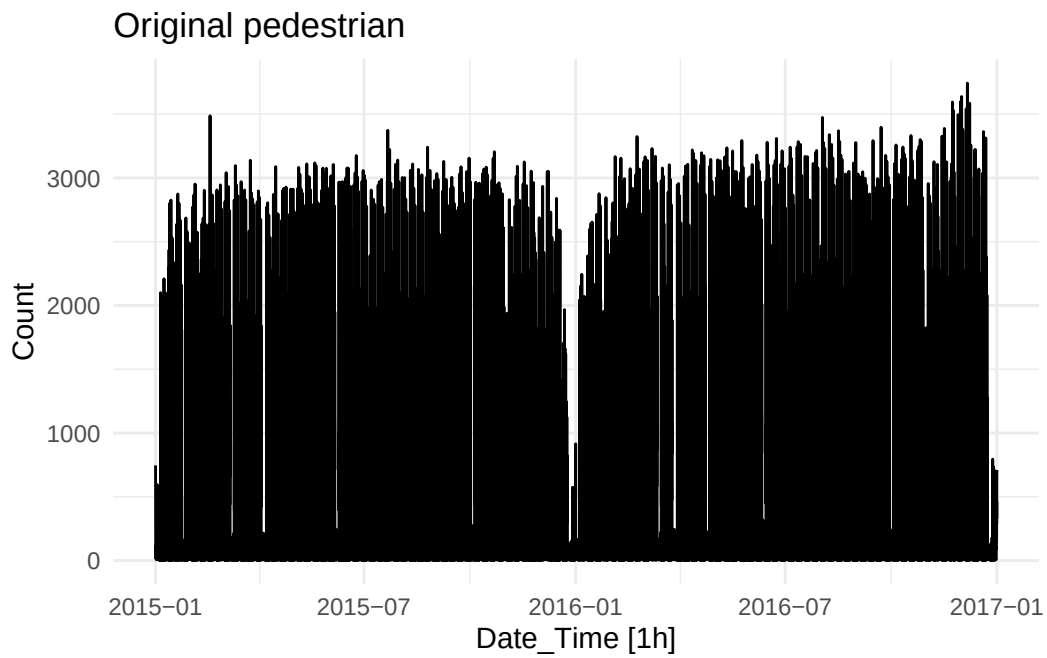
```
autoplot(aus_production, Tobacco) + labs(title="Original aus_production")
```



```
autoplot(ansett |> filter(Class == 'Economy', Airports=="MEL-SYD"), Passengers) + labs(title="Original ansett Passengers")
```



```
autoplot(pedestrian |> filter(Sensor == "Southern Cross Station"), Count) + labs(title="Orig
```




```

optim_boxcox <- function(data, feature) {
  # Use enquos to capture the feature argument
  feature_quo <- enquos(feature)

  # Calculate optimal lambda using guerrero method
  lambda <- data |>
    features(!feature_quo, features = guerrero) |>
    pull(lambda_guerrero)

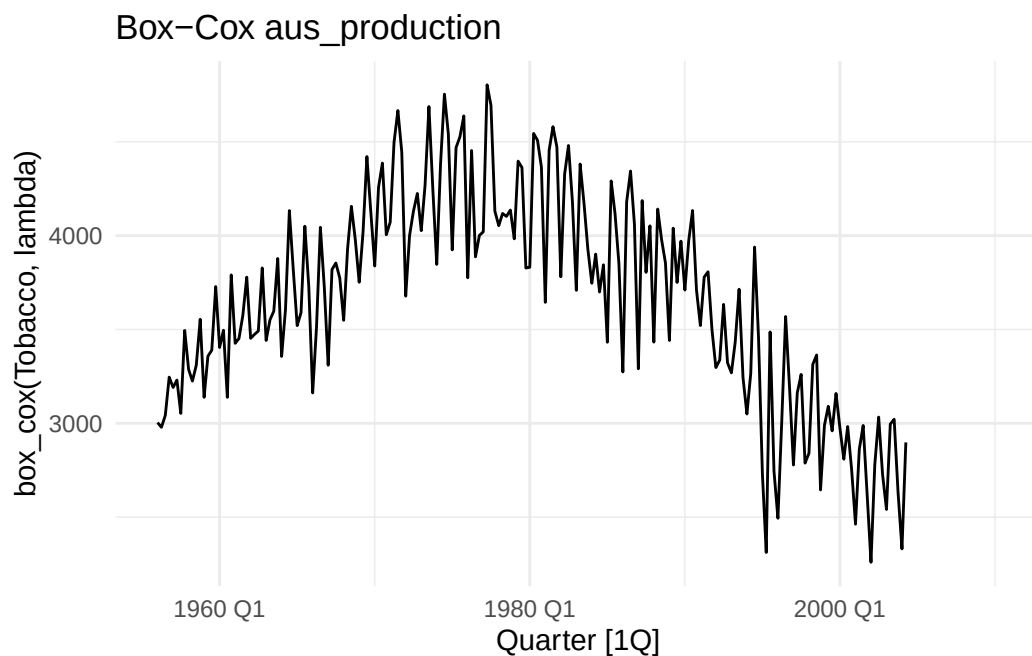
  print(paste("Optimum Lambda Value:", lambda))

  # Create plot with transformed data
  data |>
    autoplot(box_cox(!feature_quo, lambda))
}

# Call the function
optim_boxcox(aus_production, Tobacco) + labs(title="Box-Cox aus_production")

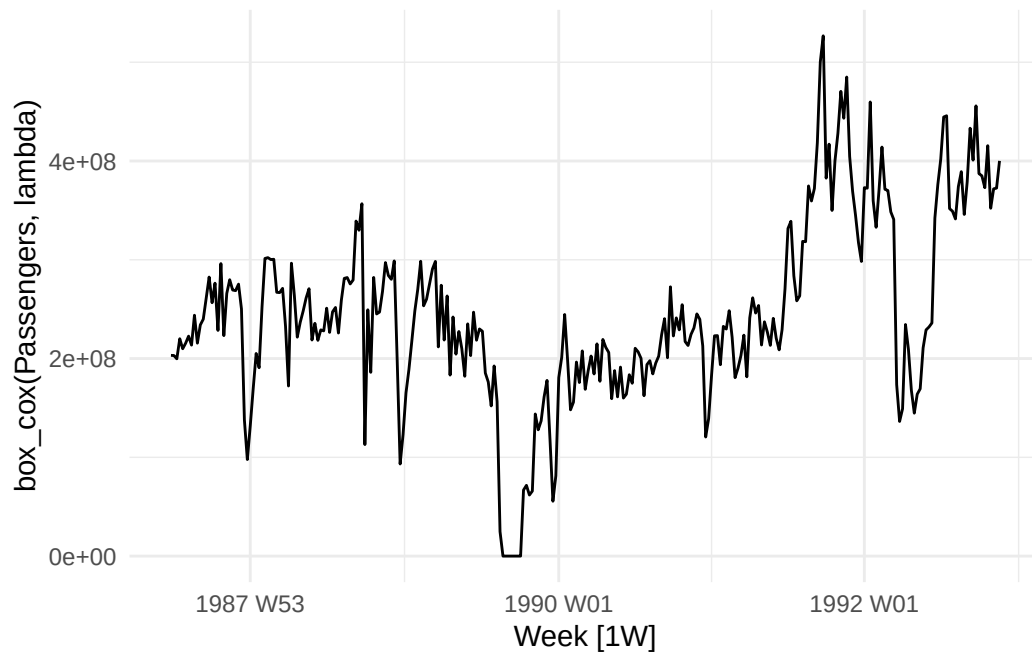
```

```
[1] "Optimum Lambda Value: 0.926463585274457"
```



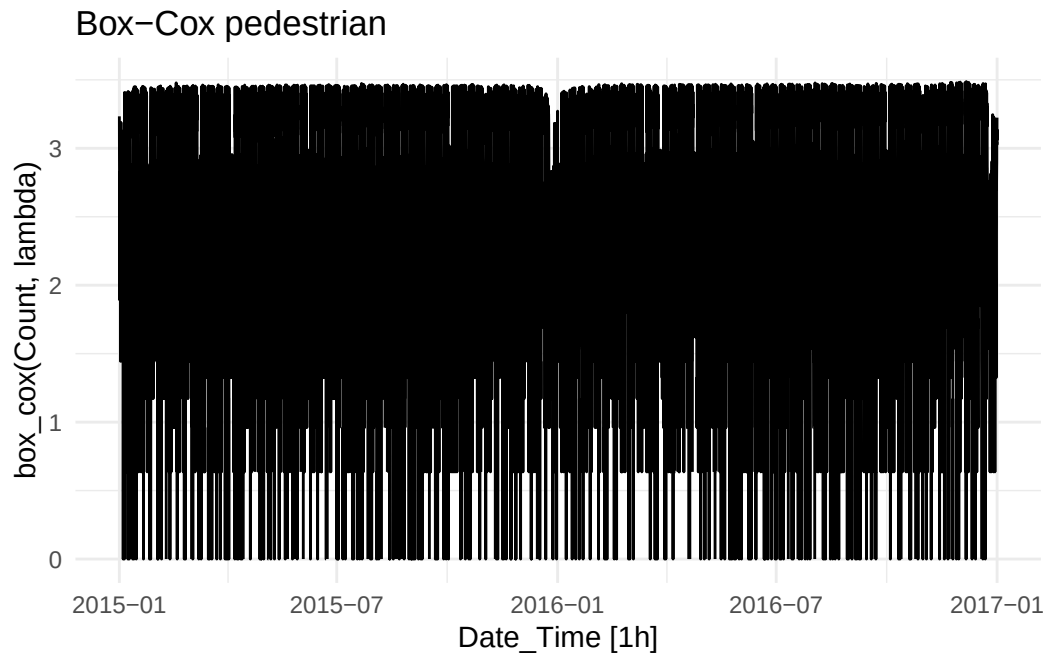
```
optim_boxcox(ansett |> filter(Class == 'Economy', Airports=="MEL-SYD"), Passengers) + labs("I
```

```
[1] "Optimum Lambda Value: 1.9999267732242"
```



```
optim_boxcox(pedestrian |> filter(Sensor == "Southern Cross Station"), Count) + labs(title="I
```

```
[1] "Optimum Lambda Value: -0.250161562391103"
```



1.7 Exercise 3.6

Show that a 3×5 MA is equivalent to a 7-term weighted moving average with weights of 0.067, 0.133, 0.200, 0.200, 0.133, and 0.067.

1.7.1 Response

I've shown below analytically that the outcome for these two computations are the same. Moving averages are effectively weighting coefficients on each period of the time series, and as long as the weights both sum to one across the same range of periods, they will be equivalent.

```
# Create a dummy series
my_series <- sample(1:20, size=20, replace=TRUE)

# Turn it into a tibble
my_ts <- tibble(
  year = 1:20, value = my_series
)

# Apply slider moving average, 3 then 5 to get 3x5
ma_ts <- my_ts |>
```

```

mutate(
  `3-MA` = slider::slide_dbl(value, mean,
    .before = 1, .after = 1, .complete = TRUE),
  `3x5-MA` = slider::slide_dbl(`3-MA`, mean,
    .before = 2, .after = 2, .complete = TRUE)
)

# Establish our weights
weights <- c(0.067, 0.133, 0.200, 0.200, 0.200, 0.133, 0.067)

# Function apply weights
ma_weighted <- function(x, weights) {
  weighted_ma <- stats::filter(x, weights, sides = 2)
  return(as.numeric(weighted_ma))
}

# Add weighted values to tibble
weight_ts <- ma_ts |>
  mutate(weighted_value = ma_weighted(my_ts$value, weights))

# Display results
weight_ts |>
  drop_na() |>
  select(year, "3x5-MA", weighted_value) |>
  kable()

```

year	3x5-MA	weighted_value
4	11.666667	11.661
5	12.400000	12.401
6	11.333333	11.329
7	10.266667	10.270
8	9.533333	9.536
9	10.400000	10.402
10	12.200000	12.200
11	13.533333	13.529
12	13.800000	13.799
13	12.266667	12.263
14	10.000000	10.001
15	7.933333	7.937
16	7.066667	7.068
17	7.800000	7.803

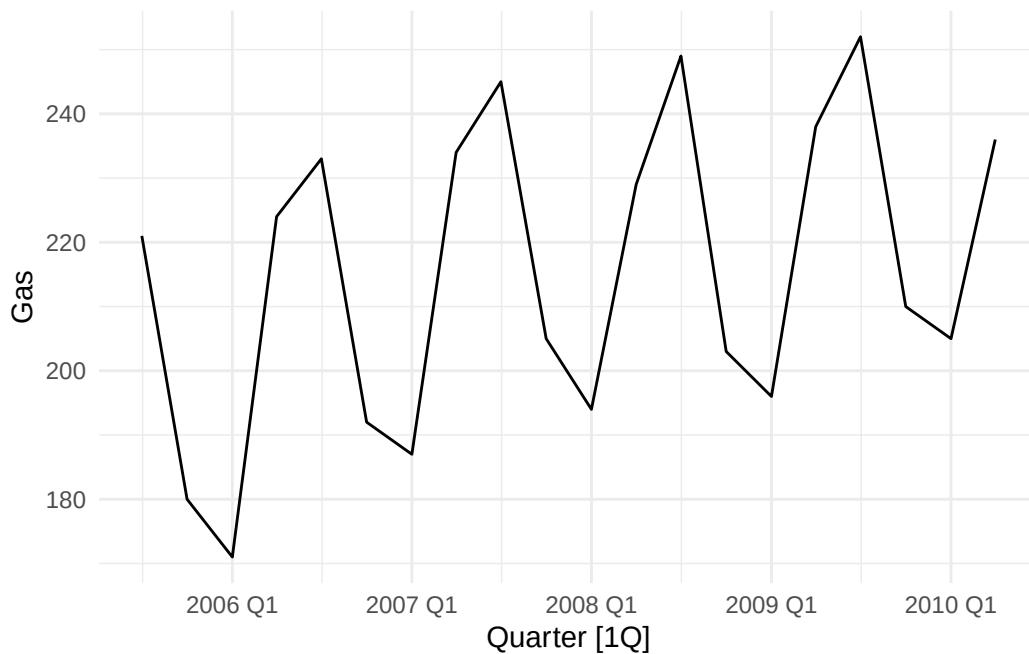
1.8 Exercise 3.7

Consider the last five years of the Gas data from `aus_production`.

1. Plot the time series. Can you identify seasonal fluctuations and/or a trend-cycle?

```
gas <- tail(aus_production, 5*4) |> select(Gas)

# Plot the time series
autoplot(gas)
```



We can definitely see seasonal peaks and valleys in the data, especially around winter months (assuming northern hemisphere).

2. Use `classical_decomposition` with `type=multiplicative` to calculate the trend-cycle and seasonal indices.

```
gas_decomp <- gas |>
  model(
    classical_decomposition(Gas, type='multiplicative')
  ) |>
  components()

kable(head(gas_decomp))
```

.model	Quarter	Gas	trend	seasonal	random	season_adjust
classical_decomposition(Gas, type = "multiplicative")	2005 Q3	221	NA	1.1256812	NA	196.3256
classical_decomposition(Gas, type = "multiplicative")	2005 Q4	180	NA	0.9250656	NA	194.5808
classical_decomposition(Gas, type = "multiplicative")	2006 Q1	171	200.50	0.8752824	0.9743916	195.3655
classical_decomposition(Gas, type = "multiplicative")	2006 Q2	224	203.50	1.0739708	1.0249227	208.5718
classical_decomposition(Gas, type = "multiplicative")	2006 Q3	233	207.00	1.1256812	0.9999313	206.9858
classical_decomposition(Gas, type = "multiplicative")	2006 Q4	192	210.25	0.9250656	0.9871717	207.5529

3. Do the results support the graphical interpretation from part a?

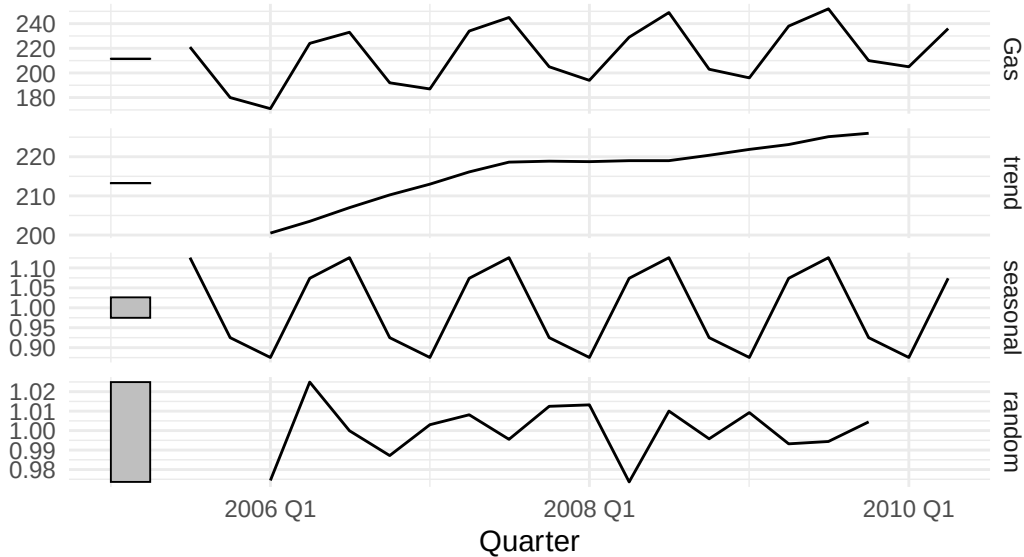
The seasonal component of the time series clearly shows regular seasonal changes. This supports our early “eyeballed” assessment.

4. Compute and plot the seasonally adjusted data.

```
gas |>
  model(
    classical_decomposition(Gas, type='multiplicative')
  ) |>
  components() |>
  autoplot()
```

Classical decomposition

Gas = trend * seasonal * random

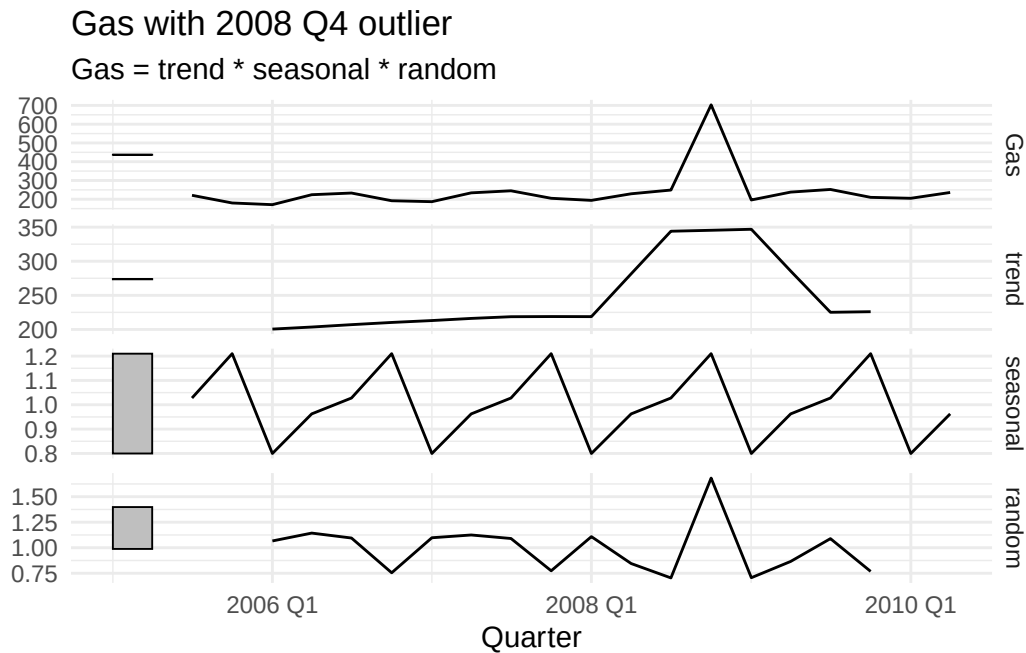


5. Change one observation to be an outlier (e.g., add 300 to one observation), and recompute the seasonally adjusted data. What is the effect of the outlier?

The outlier affects the trend and random components, but the seasonal component is largely unaffected. This is because we only observe the outlier once, so it is not “picked up” as a repeating seasonal trend.

```
gas_outlier <- gas |>
  mutate(
    Gas = ifelse(Quarter == yearquarter('2008 Q4'), Gas + 500, Gas)
  )

gas_outlier |>
  model(
    classical_decomposition(type="multiplicative")
  ) |>
  components() |>
  autoplot() + labs(title="Gas with 2008 Q4 outlier")
```



6. Does it make any difference if the outlier is near the end rather than in the middle of the time series?

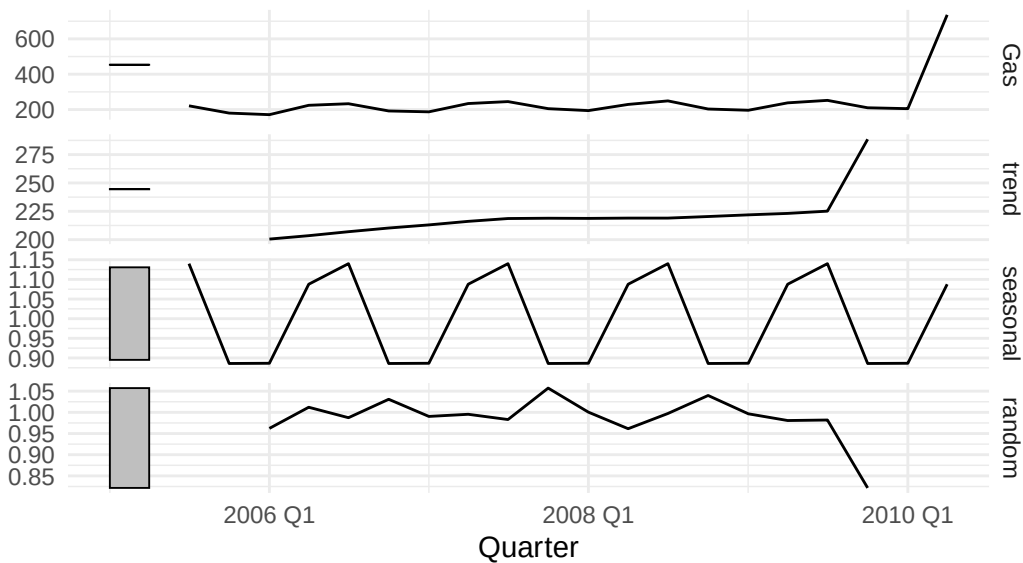
Adding the outlier to the end of the series has still not affected the seasonality, for the same reasons mentioned above.

```
gas_late_outlier <- gas |>
  mutate(
    Gas = ifelse(Quarter == yearquarter('2010 Q2'), Gas + 500, Gas)
  )

gas_late_outlier |>
  model(
    classical_decomposition(type="multiplicative")
  ) |>
  components() |>
  autoplot() + labs(title="Gas with 2010 Q2 outlier")
```


Gas with 2010 Q2 outlier

Gas = trend * seasonal * random



1.9 Exercise 3.8

Recall your retail time series data (from Exercise 7 in Section 2.10). Decompose the series using X-11. Does it reveal any outliers, or unusual features that you had not noticed previously?

1.9.1 Response

Decomposing the time series using X-11 allows us to see two important things.

Firstly, there are several positive outliers near January 2010 in the residual component, where we can see spikes of similar magnitude to the regular seasonality - this was not obvious from the original graph.

Secondly, We can see that the seasonality trend is widening as time goes on, meaning the seasonality is becoming slightly more gradual.

```
set.seed(42)

myseries <- tsibbledata::aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

myseries |>
```

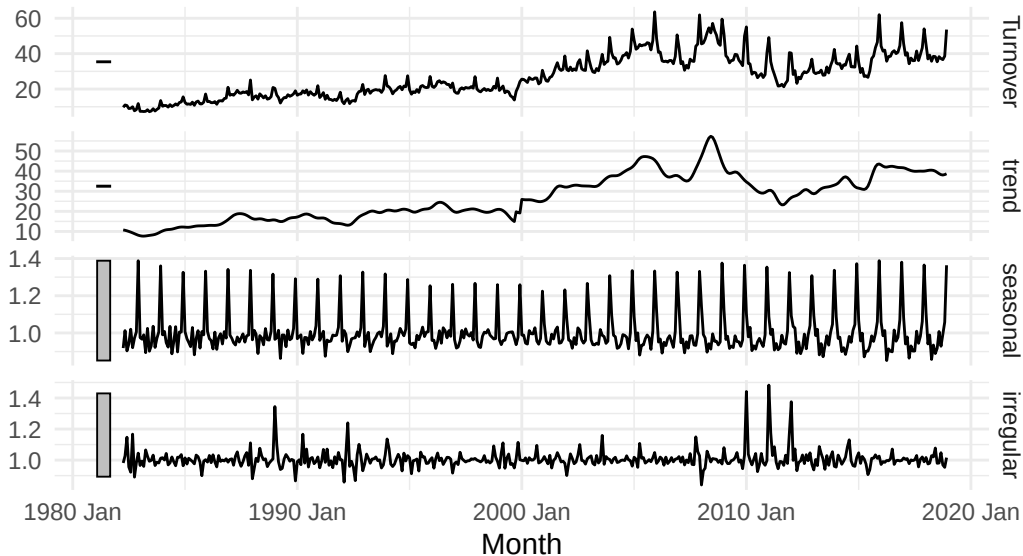
```

model(
  x11 = X_13ARIMA_SEATS(Turnover ~ x11())
) |>
components() |>
autoplot()

```

X-13ARIMA-SEATS using X-11 adjustment decomposition

Turnover = trend * seasonal * irregular



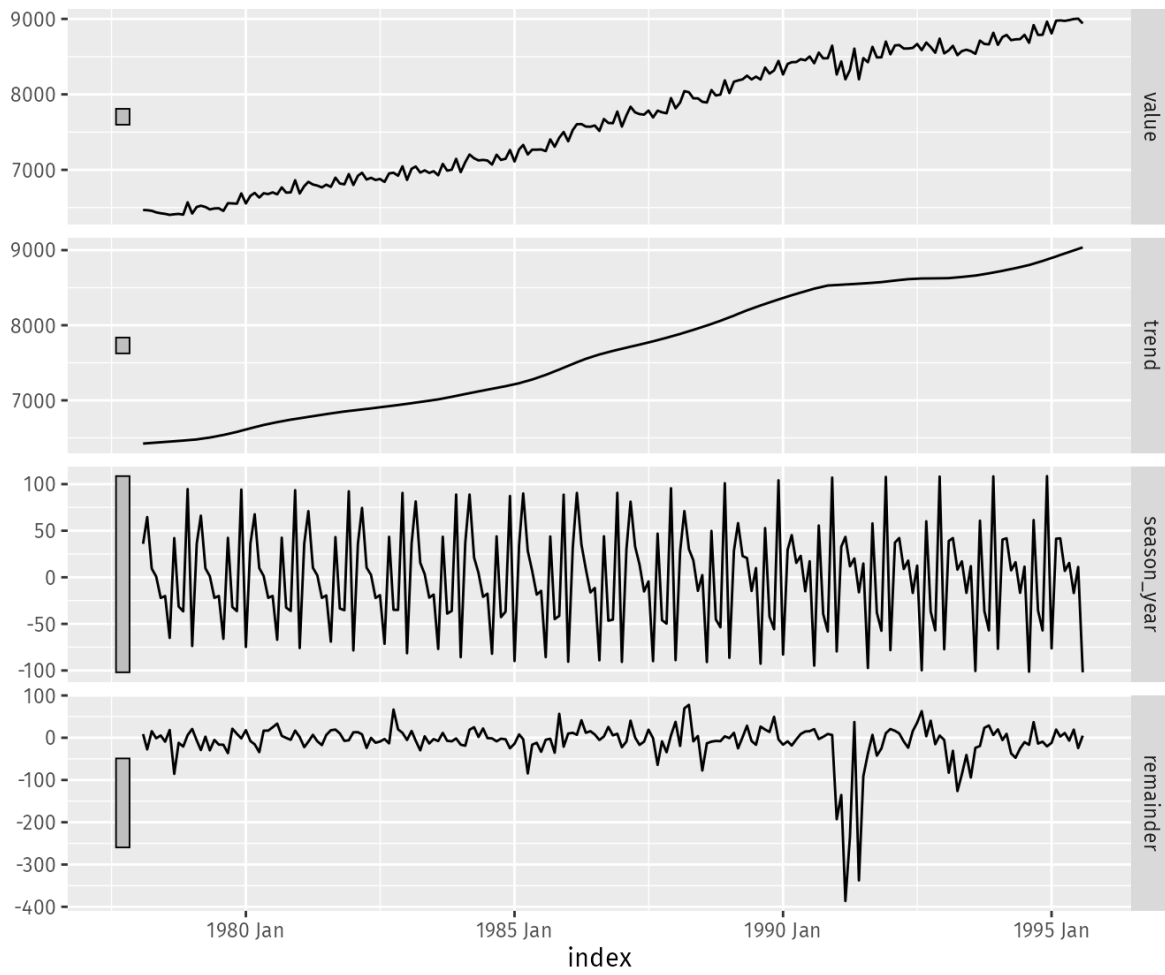
1.10 Exercise 3.9

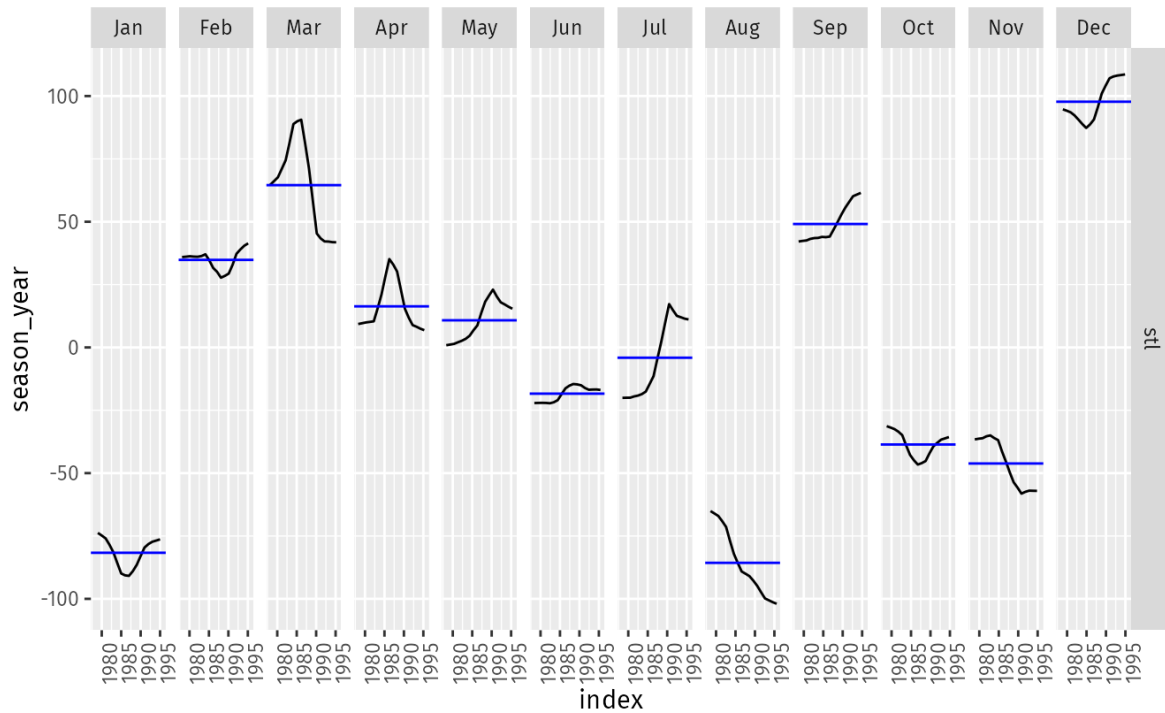
Figures 3.19 and 3.20 show the result of decomposing the number of persons in the civilian labour force in Australia each month from February 1978 to August 1995.

Write about 3–5 sentences describing the results of the decomposition. Pay particular attention to the scales of the graphs in making your interpretation.

STL decomposition

value = trend + season_year + remainder





The decomposition reveals that the largest component of this series is trend. Seasonality has been extracted, but plays a much smaller role overall, orders of magnitude smaller than the trend component (only ± 50 on a scale of thousands). In fact, the seasonal component is the smallest one; even the remainder plays a larger role.

Is the recession of 1991/1992 visible in the estimated components?

Yes, but only in the remainder component. We would not expect to see something like a single recession in the seasonality trend, given it only occurs once